Lecture 6 - Analysis and Efficiency

Computer systems, data structures, and data management (4CM508)

Dr Sam O'Neill

Experimental Analysis

We can study the running time of any algorithm by timing its execution time.

```
import time
start = time.perf_counter()     # start the timer

# run the code to be timed here

finish = time.perf_counter()    # stop the timer
elapsed_time = finish - start   # calculate the elasped time
```

If we plot this for different-sized inputs, we can understand how it behaves.

Demonstration

We are going to look at the prefix (cumulative) average.

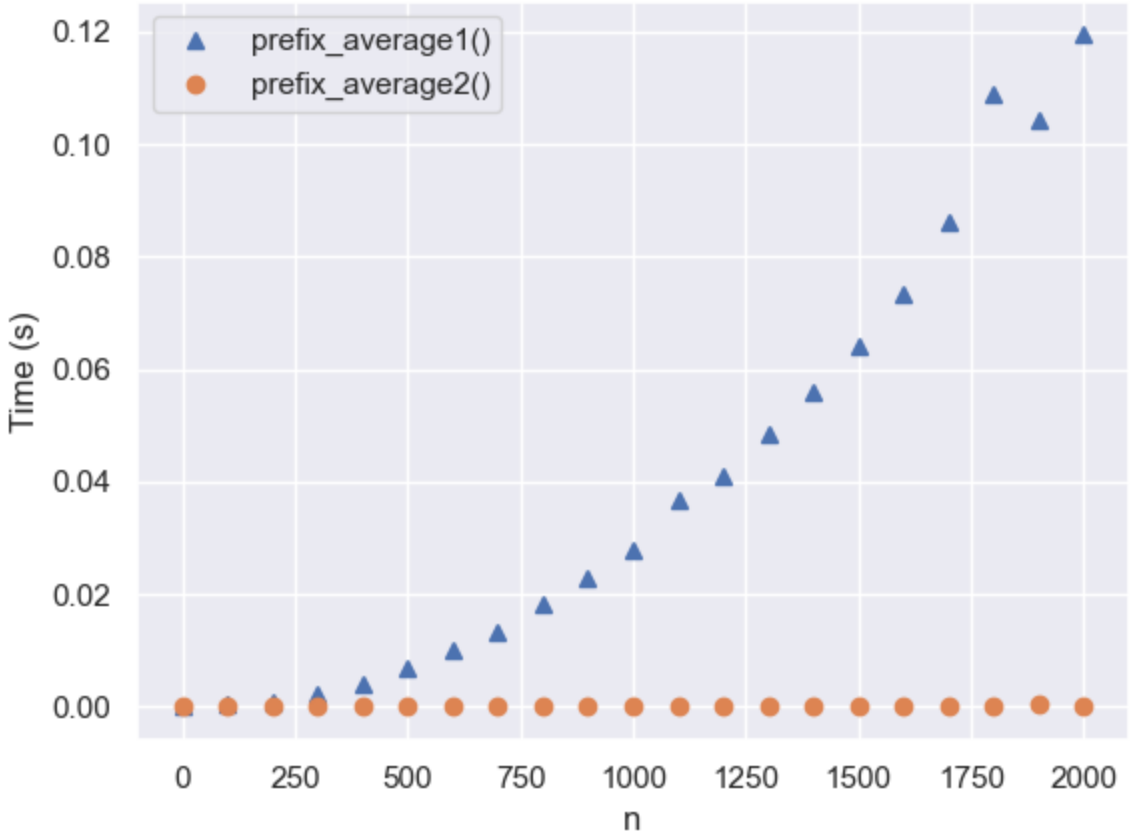
Let's see how to calculate this via an animation.

Python Implementations

Prefix Average 1

Prefix Average 2

Experimental Analysis of Prefix Average



Issues With Experimental Analysis

- We can't test for all possible inputs (it could be that some of the values we have missed out are important).
- Comparing algorithms requires us to run experiments using the same hardware and software.
- An algorithm has to be implemented to be tested!

Beyond Experimental Analysis

What we need is a way to analyse algorithms that:

1. Does not require the implementation of the algorithm.
2. Analysis is independent of hardware and software.
3. We consider all inputs.

This is the purpose of this lecture. We will look at how we can analyse an algorithm by counting the number of ***basic operations*** it takes.

Model of Computation

The following will be ***basic operations*** that will take constant time:

1. Arithmetic operations such as `+,-,\,*`
2. Comparisons operations such as `<,>,<=,>=, ==`
3. Boolean operations such as `and`, `or` and `not`
4. Reading and writing a word to memory

We donate constant time by $O(1)$. This will make more sense later on.

Worse Case Analysis

- The **worst-case time complexity** indicates the **longest running time** performed by an algorithm **given input of size** n
  - Guarantees that the algorithm will finish in the indicated period of time.

Average Case Analysis

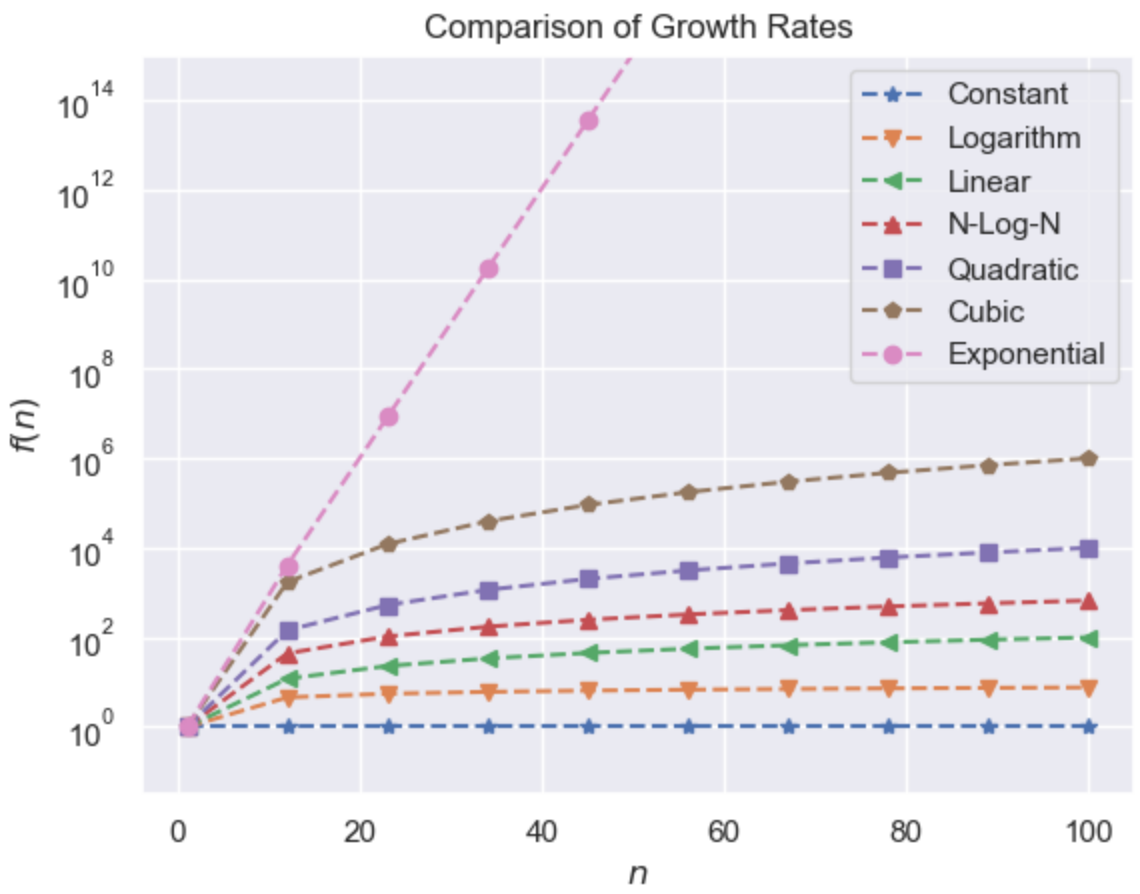
In general, we will focus on worse-case analysis. Computing average-case analysis is much more complicated.

Growth Rates

These progressively get worse.

Name	$f(n)$
Constant	$f(n) = c$
Logarithm	$f(n) = \log_b(n)$ most of the time we will use $\log(n) = \log_2(n)$
Linear	$f(n) = n$
N-Log-N	$f(n) = n\log(n)$
Quadratic	$f(n) = n^2$
Polynomial	$f(n) = a_0 + a_1 n + a_2 n^2 + a_3 n^3 + \cdots + a_d n^d$ where $a_0, a_1, \ldots a_d$ are constants and d is the highest power of n.
Exponential	$f(n) = b^n$ where b is a positive constant.

Comparison of Growth Rates



Interactive Growth Rate Plot

Big-O

- Big-O notation is a way of providing an upper bound on the number of ***basic operations*** an algorithm performs based on an input parameter n.
  - n is always the size of the input.
- For example, if we are looking for the maximum item in a list of numbers. n is the length of the list.
- The idea is that we count the number of ***basic operations*** and then look at the maximum growth rate.

Formal Definition

Let $f(n)$ and $g(n)$ be functions that map positive integers to real numbers, i.e. $f : \mathbb{Z} \to \mathbb{R}$.

$f(n)$ is $O(g(n))$ if there is a real constant c and an integer constant $n_0 \geq 1$ such that

$$f(n) \leq cg(n), \quad \text{for } n \geq n_0$$

What does that even mean!?

In plain English, the function $f(n)$ is said to be $O(g(n))$ (big-O $g(n)$) if for a big enough value of n (we call that n_0) the function $f(n)$ is always smaller than $cg(n)$.

Example

The function $10n + 6$ is $O(n)$.

We have to find a constant $c > 0$ and $n_0 \geq 1$ such that

$$10n + 6 \leq cn, \quad \text{for } n \geq n_0$$

Pick $c = 11$ and $n_0 = 6$.

We are stating

$$10n + 6 \leq 11n, \quad \text{for } n \geq 6$$

Is this true?

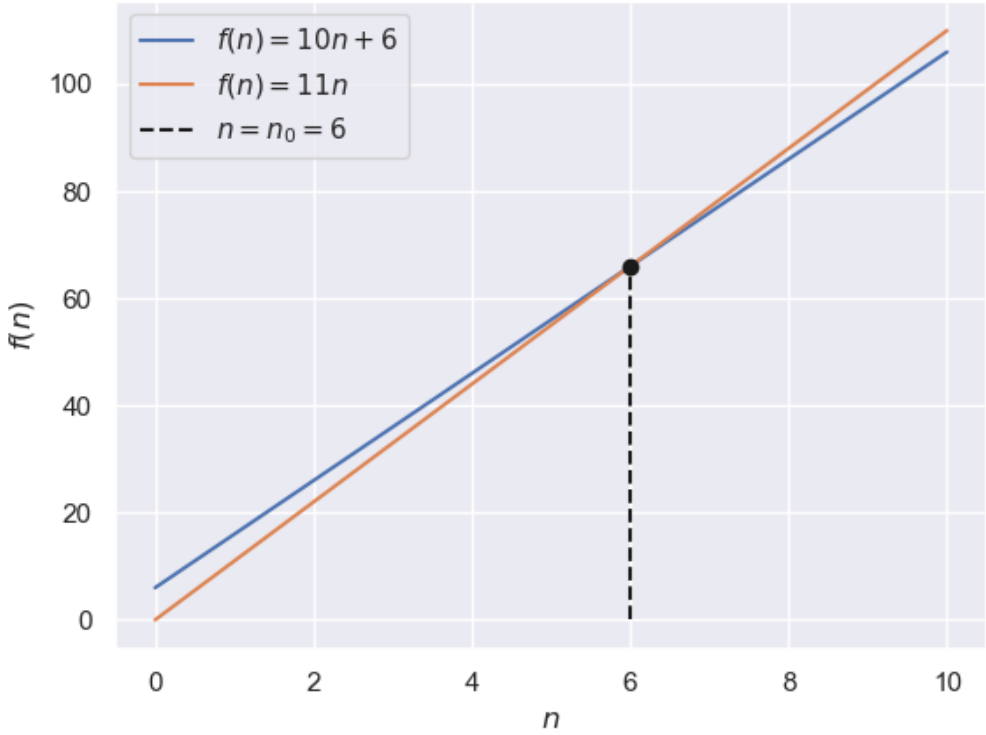
Try for $6, 7, 8, \ldots$

Example Continued...

Table of n values

n	$10n + 6$	$11n$	$10n + 6 \leq 66$
5	56	55	`False`
6	66	66	`True`
7	76	77	`True`
8	86	88	`True`

Plot of $10n + 6$ and $11n$



Easier Way

Let's say we counted the total number of ***basic operations*** in an algorithm to be

$$3n^2 + 2n$$

then n^2 is the dominant term.

By that, we mean that as n gets bigger and bigger, i.e. $n \to \infty$, then n^2 is the term that affects the total the most.

Therefore we say that the worse case running time is $O(n^2)$.

Some Other Examples

Number of Basic Operations	Big-O Running Time
$4n + 3$	$O(n)$
$5n^2 + 4$	$O(n^2)$
$5n^4 + 34n^2 + 4$	$O(n^4)$
$2^n + 3n^4 + n^2$	$O(2^n)$
$n\log(n) + 5n$	$O(n\log(n))$

Worse Case Analysis Example 1

```
def my_max(data):
    largest = data[0]           # Initial item
    for item in data:           # For each item
        if item > largest       # If it is better than largest
            largest = item      # set new largest
    return largest
```

Worse Case Analysis Example 2

```
def prefix_average1(S):
    """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
    n = len(S)
    A = [0]*n                    # create new list of n zeros
    for j in range(n):
        total = 0                # begin computing S[0] + ... + S[j]
        for i in range(j + 1):
            total += S[i]
        A[j] = total / (j+1)     # record the average
    return A
```

Worse Case Analysis Example 3

```
def prefix_average2(S):
    """Return list such that, for all j, A[j] equals average of S[0], ..., S[j]."""
    n = len(S)
    A = [0]*n                    # create new list of n zeros
    total = 0                    # compute prefix sum as S[0] + S[1] + ...
    for j in range(n):
        total += S[j]            # update prefix sum to include S[j]
        A[j] = total / (j+1)     # compute average based on current sum
    return A
```

Summary

- We can count the number of ***basic operations*** an algorithm performs.
- We can use worse-case analysis to understand the running time of an algorithm based on the input size n.
- We use big-O notation to provide an upper bound on this worse-case analysis.
- NOTE: We can also use big-O notation for average-case analysis and best-case analysis.

Key Takeaway

If someone tells you an algorithm has a worse-case complexity of $O(n)$, this is better than another algorithm with a worse-case complexity of $O(n^2)$.

$O(1) < O(\log(n)) < O(n) < O(n\log(n)) < O(n^2) < O(2^n)$.